MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Dulgheru Ion

Report

Practical Work No. 8
Operating Systems

Topic: Processes Talking Through Pipes, in xv6

Checked by: _____

Chişinău – 2026

TABLE OF CONTENTS

1 AIM OF THE WORK 3

1.1 Purpose of practical work 3

1.2 Problem statement 3

2 THEORETICAL BACKGROUND 4

2.1 Processes in xv6 4

2.2 Pipes 4

2.3 Concurrent prime sieve 4

2.4 Resource limits of xv6 4

3 DESCRIPTION OF EXERCISES 6

3.1 Exercise 1: pingpong 6

3.2 Exercise 2: primes 8

3.3 Exercise 3: experiment with the limit 280 10

CONCLUSION 14

REFERENCES 15

1 AIM OF THE WORK

1.1 Purpose of practical work

The purpose of the practical work is to use the process and pipe system calls of the xv6 operating system (fork, pipe, read, write, close, wait, exit and getpid) in order to build programs in which several processes cooperate through pipes [1]. The work also aims to observe how the limits of the kernel bound a long chain of such processes.

The practical work pursues the following goals:

- to implement the pingpong program, in which two processes exchange one byte through two pipes;
- to implement the primes program, a concurrent prime sieve built as a chain of processes;
- to run primes with the upper limit raised from 35 to 280 and to analyse the observed failure;
- to document the runs with screenshots and to answer the observation questions of the handout.

1.2 Problem statement

Two programs are required, both taken from the util lab of the MIT 6.1810 course [2]. The first program, pingpong, must create two pipes and a child process: the parent sends one byte to the child, the child prints the line ": received ping", returns the byte to the parent, and the parent prints ": received pong" and waits for the child to finish. The line "ping" must appear first and must be printed by the child.

The second program, primes, must print the prime numbers from 2 to 35 using a chain of processes. Each process reads its first number p, prints it as a prime, creates a pipe and a child process, and forwards to the child only the numbers that are not divisible by p. When the input ends, the process closes its output, waits for the child and exits. The expected output consists of eleven lines, from "prime 2" to "prime 31".

Finally, the handout asks three observation questions for each program, including the behaviour of primes when the limit is raised to 280. The answers are given in chapter 3.

2 THEORETICAL BACKGROUND

2.1 Processes in xv6

In xv6 a new process is created with the fork call, which duplicates the calling process [3]. The call returns 0 in the child and the PID of the child in the parent, so the two copies can follow different branches of the same program. The wait call blocks the parent until a child terminates, and exit ends the calling process.

Every process owns a table of open file descriptors. The fork call copies this table, therefore the descriptors of the parent and of the child refer to the same open files, and a pipe end remains open until all descriptors that refer to it are closed.

2.2 Pipes

A pipe is a kernel buffer with a read end and a write end, created by the pipe call, which returns two descriptors [3]. The pipe is a single unidirectional byte stream. A read on an empty pipe blocks until data arrives or until every write end is closed, in which case read returns 0. A write on a full pipe blocks until the reader consumes data. The buffer of an xv6 pipe holds 512 bytes.

Two consequences follow. First, two processes that must exchange data in both directions need two pipes. Second, each process must close the pipe ends that it does not use, otherwise the end of the input is never detected.

2.3 Concurrent prime sieve

The sieve designed by Doug McIlroy, the inventor of the UNIX pipe, uses one process per prime number [4]. The first process feeds the integers into a pipe. Each stage keeps the first number received as a prime and passes on only the numbers that are not its multiples, so every composite number is removed by the stage of its smallest prime factor. The stages run concurrently, and the pipes provide the synchronisation.

2.4 Resource limits of xv6

The kernel of xv6 fixes several static limits in the file kernel/param.h [5]. The values relevant to this work are listed in table 2.1.

Table 2.1 Resource limits defined in kernel/param.h

Constant	Value	Meaning
NPROC	64	maximum number of processes
NOFILE	16	open files per process
NFILE	100	open files per system

Each pipe occupies two entries of the system-wide file table, one for the read end and one for the write end. According to formula (2.1), the number of pipes that can exist at the same time is bounded as follows:

$$N_{pipe} = NFILE / 2 = 100 / 2 = 50 \quad (2.1)$$

where N_{pipe} is the maximum number of simultaneously open pipes, and NFILE is the size of the system-wide file table. The entries used by other open files, for example the console, reduce this number slightly.

3 DESCRIPTION OF EXERCISES

3.1 Exercise 1: pingpong

Objective

The objective is to exchange one byte between a parent and a child process through two pipes, one for each direction.

System calls and files used

- system calls: pipe, fork, read, write, close, getpid, wait and exit;
- source file: user/pingpong.c;
- build file: Makefile, where the entry $U/_pingpong\ is added to the UPROGS list.

Description

Before the work started, the xv6 tree was checked: the system boots up to the shell prompt, and the search in user/user.h shows that the call sleep() was renamed to pause(), as presented in figure 3.1. This change does not affect the present work, because neither sleep nor pause is used.

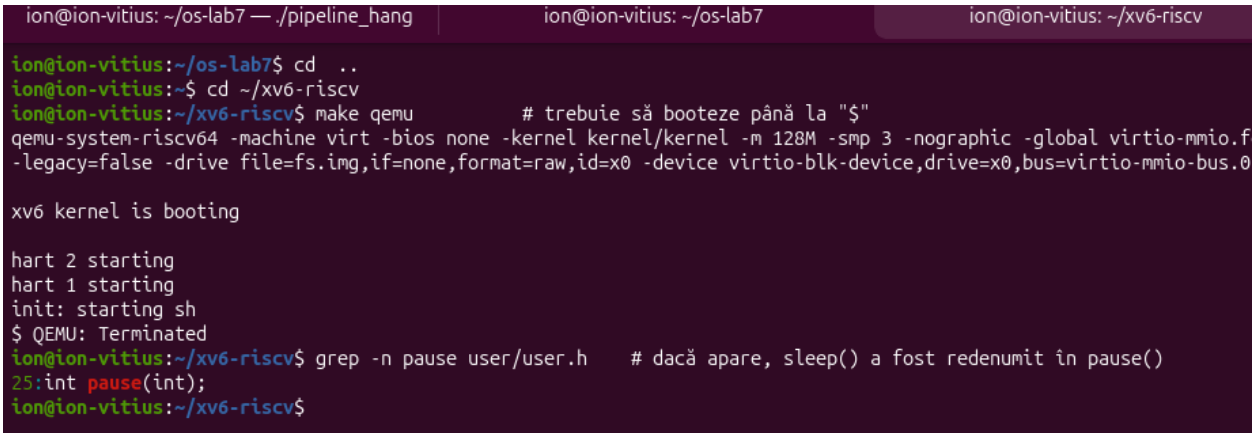


Figure 3.1 Boot check of the xv6 tree and search for pause() in user/user.h

The program was written in the file user/pingpong.c, whose source code is given below. The array p2c carries data from the parent to the child, and the array c2p carries data from the child to the parent.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];   // parent->child, child->parent
  char b = 'x';

  if(pipe(p2c) < 0 || pipe(c2p) < 0){
    fprintf(2, "pingpong: pipe failed\n");
    exit(1);
  }
  int pid = fork();
  if(pid < 0){
    fprintf(2, "pingpong: fork failed\n");
    exit(1);
  }
  if(pid == 0){
    // child: close the unused ends
    close(p2c[1]);
    close(c2p[0]);
```

```
    if(read(p2c[0], &b, 1) != 1)
      exit(1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);
    close(p2c[0]);
    close(c2p[1]);
    exit(0);
  }
  // parent
  close(p2c[0]);
  close(c2p[1]);
  write(p2c[1], &b, 1);
  read(c2p[0], &b, 1);
  printf("%d: received pong\n", getpid());
  close(p2c[1]);
  close(c2p[0]);
  wait(0);
  exit(0);
}
```

A program becomes available in xv6 only after it is added to the UPROGS list of the Makefile. The two new entries, $U/_pingpong\ and $U/_primes\, were inserted after the entry $U/_zombie\. According to figure 3.2, each entry starts with a TAB character (shown as ^I) and ends with a backslash, and the pattern pingpong occurs exactly once in the file.

```
EOF
ion@ion-vitius:~/xv6-riscv$ sed -n '146,152p' Makefile | cat -A
grep -c pingpong Makefile
^I$U/_grind\$
^I$U/_wc\$
^I$U/_zombie\$
^I$U/_pingpong\$
^I$U/_primes\$
^I$U/_sleep\$
^I$U/_args\$
1
ion@ion-vitius:~/xv6-riscv$
```

Figure 3.2 UPROGS entries in the Makefile, displayed with cat -A

The result of the execution is presented in figure 3.3. The child, with PID 4, prints the line “received ping” first, and the parent, with PID 3, prints the line “received pong” afterwards.

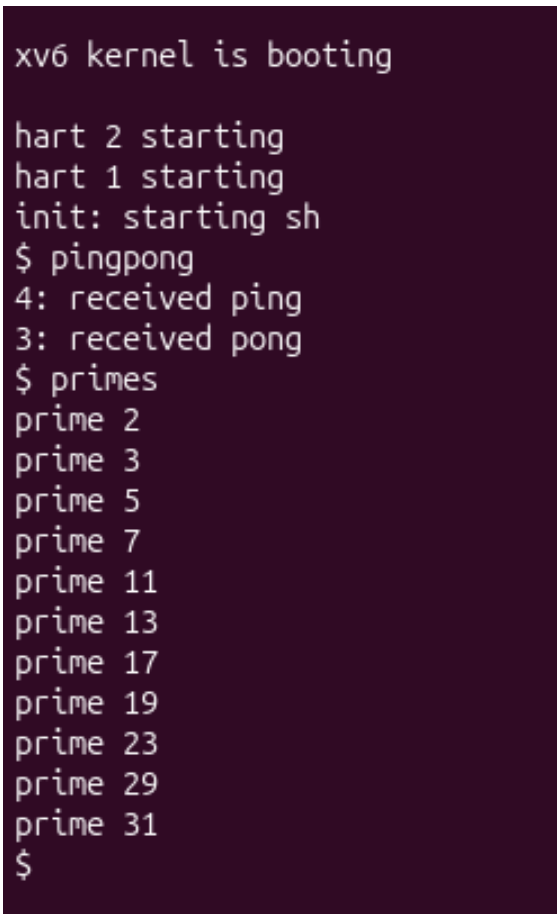


Figure 3.3 Output of the programs pingpong and primes in xv6

The observation questions of the handout are answered as follows:

a) two pipes are needed because a pipe is a single stream without direction, so with one pipe the writing process could read back its own byte before the other process takes it;

b) the parent waits because read on the empty pipe c2p blocks the process until the child writes the reply, and the final wait(0) additionally waits for the termination of the child;

c) the child has the larger PID because PIDs are assigned in increasing order and the child is created after the parent (in the run above, 4 and 3).

3.2 Exercise 2: primes

Objective

The objective is to print the prime numbers from 2 to 35 with a chain of processes connected by pipes, each process filtering the multiples of its own prime.

System calls and files used

- system calls: pipe, fork, read, write, close, wait, exit and printf;
- source file: user/primes.c;
- build file: Makefile, entry $U/_primes\.

Description

The source code of the program is given below. The generator, which is the function main, writes the integers 2 to 35 into the first pipe as 4-byte values. Every call of the function stage reads the first number p, prints it, creates the next pipe and the next process, and forwards the numbers that are not divisible by p.

```
#include "kernel/types.h"
#include "user/user.h"

void stage(int in) __attribute__((noreturn));

void
stage(int in)
{
  int p, n;
  int fd[2];

  if(read(in, &p, sizeof(p)) != sizeof(p)){
    close(in);
    exit(0);
  }
  printf("prime %d\n", p);

  if(pipe(fd) < 0){
    printf("primes: pipe failed after prime %d\n", p);
    close(in);
    exit(1);
  }
  int pid = fork();
  if(pid < 0){
    printf("primes: fork failed after prime %d\n", p);
    close(in);
    close(fd[0]);
    close(fd[1]);
    exit(1);
  }
  if(pid == 0){
    close(fd[1]);     // the child only reads
    close(in);        // the parent's input is not needed
    stage(fd[0]);
  }
  close(fd[0]);       // the parent only writes
  while(read(in, &n, sizeof(n)) == sizeof(n)){
    if(n % p != 0)
      write(fd[1], &n, sizeof(n));
  }
  close(in);
  close(fd[1]);       // signals EOF to the next stage
  wait(0);
  exit(0);
}

int
main(int argc, char *argv[])
{
  int fd[2];
  if(pipe(fd) < 0){
    fprintf(2, "primes: pipe failed\n");
    exit(1);
  }
  if(fork() == 0){
    close(fd[1]);
    stage(fd[0]);
  }
  close(fd[0]);
  for(int i = 2; i <= 35; i++)
    write(fd[1], &i, sizeof(i));
  close(fd[1]);
```

```
  wait(0);
  exit(0);
}
```

The three traps mentioned in the handout are handled in the code. The function stage is declared with the attribute noreturn, which prevents the error “infinite recursion detected” produced by the option -Werror. Each process closes the pipe ends it does not use, including the inherited input end in the child. Finally, the output end fd[1] is closed before wait(0), so that the next stage receives the end of the input and terminates.

The output of the program is shown in figure 3.3: eleven lines, from “prime 2” to “prime 31”, followed by the shell prompt. The return of the prompt shows that all processes terminated, therefore no pipe end remained open. The observation questions are answered as follows:

a) at the moment when “prime 31” is printed there are 11 stages (one for each prime from 2 to 31) and the generator, that is 12 processes of the program, or 14 processes if the shell and the init process are counted;

b) the experiment with the limit 280 is described in section 3.3;

c) the program is a sieve because each stage removes the multiples of its prime, and it runs in parallel because all stages exist simultaneously and filter the numbers as they arrive through the pipes, without waiting for the previous stage to finish.

3.3 Exercise 3: experiment with the limit 280

Objective

The objective is to determine whether primes still works when the upper limit is raised from 35 to 280, and to identify the limit of the system that stops it.

Description

The three candidate limits are NOFILE (16 open files per process), NPROC (64 processes) and NFILE (100 open files per system). Their values were checked in the source tree, as shown in figure 3.4.

```
ion@ion-vitius:~/xv6-riscv$ grep -n "NFILE\|NPROC\|NOFILE" kernel/param.h
1:#define NPROC        64                  // maximum number of processes
3:#define NOFILE       16                  // open files per process
4:#define NFILE        100                 // open files per system
ion@ion-vitius:~/xv6-riscv$
```

Figure 3.4 Limits defined in kernel/param.h

The constant in the loop of user/primes.c was changed from 35 to 280 and the program was run. The end of the output is shown in figure 3.5. The last printed prime is 269, which is the 57th prime number, and no message “primes: pipe failed” or “primes: fork failed” appears. Instead, the kernel reports usertrap(): unexpected scause 0xf for the process with pid=61, with stval=0x2ff8.

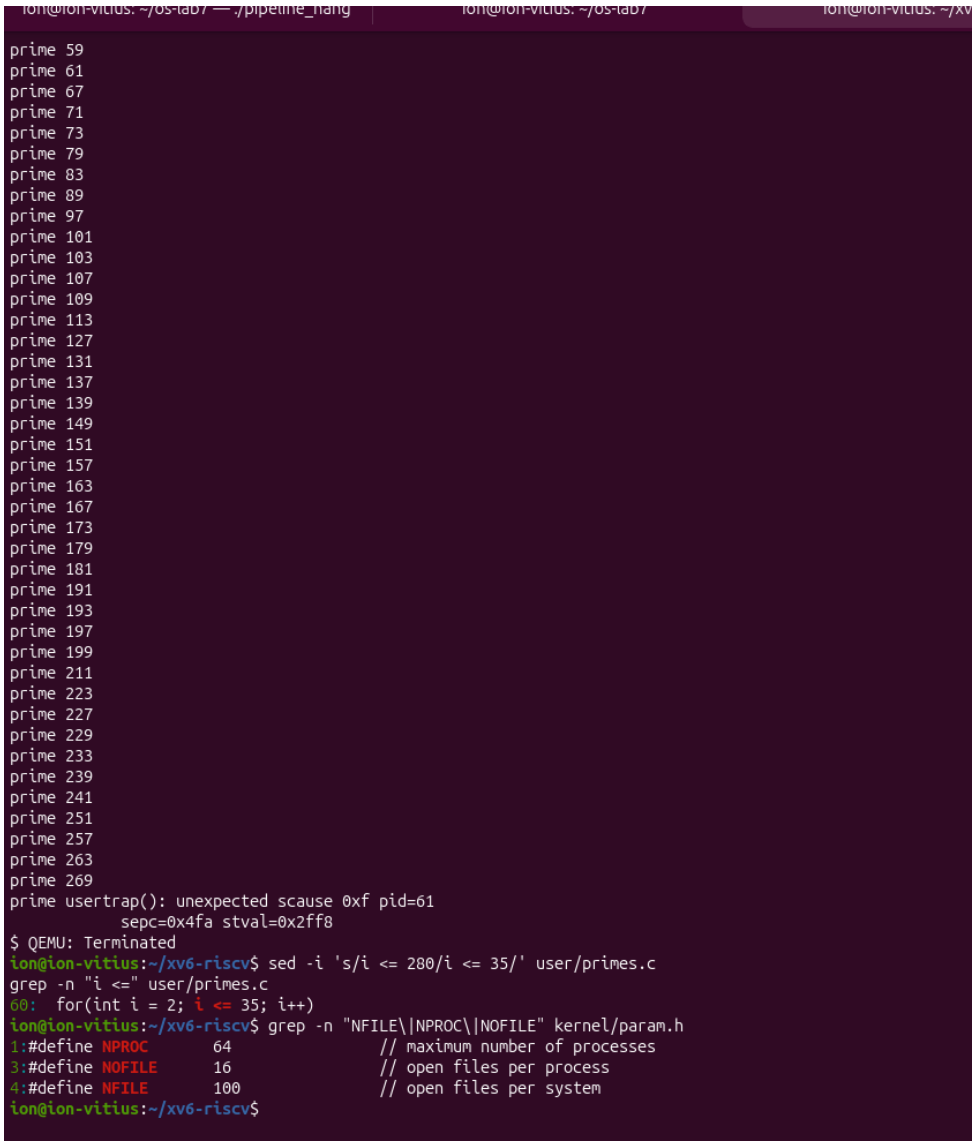


Figure 3.5 End of the output of primes with the limit 280

To identify the number of the stage that fails, the output line was extended with the PID of the process, using the call getpid(). According to figure 3.6, the last stage that printed a prime has PID 62 (prime 269), and the process that was terminated by the kernel has PID 63. The PIDs differ from the first run because the numbering depends on the number of processes created earlier in the session.

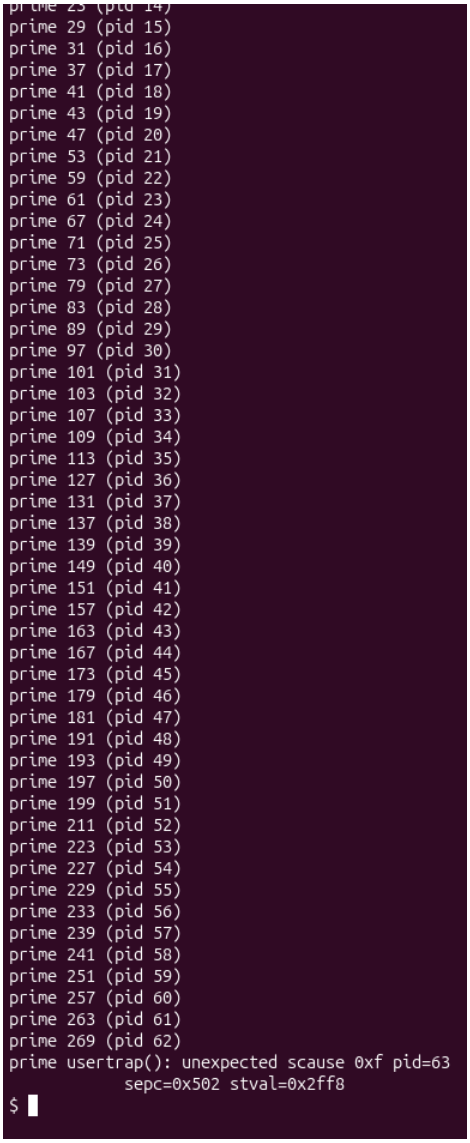


Figure 3.6 Output of primes with the PID added, limit 280

The measured values are compared with the reference behaviour described in the handout in table 3.1.

Table 3.1 Comparison between the reference behaviour and the measured behaviour

Quantity	Reference [1]	Measured
Last prime printed	227	269
Number of stages that printed a prime	49	57
Terminating event	“primes: pipe failed after prime 227”	usertrap(): unexpected scause 0xf, pid=61 (pid=63 in the run with getpid)
Value returned by pipe() and fork()	pipe() returns -1	no failure message printed

The data allows the following conclusions:

a) the program does not work completely with the limit 280;

b) pipe() and fork() did not return an error, because the corresponding messages were not printed, so the table NFILE reported by the reference was not shown to be exhausted in this run;

c) the number of live processes is at most 57 stages, the generator, the shell and the init process, that is 60, which is below NPROC = 64;

d) each stage keeps at most three descriptors open, which is far below NOFILE = 16.

Consequently, none of the three limits named in the handout was confirmed as the cause by the measurements. The failing process terminated with a store page fault (scause 0xf) at the address 0x2ff8, which means that it wrote to an address that is not mapped. The next prime after 269 is 271, so the terminated process is the 58th stage, which fails before or while printing its prime.

A possible explanation, not confirmed by a measurement, is the overflow of the user stack. In the program, a child continues inside the call stage() of its parent, with a copy of the whole stack of the parent, and then calls stage() again. The stack depth therefore grows by one frame for each created stage, regardless of how many processes are alive at that moment, and the user stack of xv6 has the size of a single page of 4096 bytes [3]. The address 0x2ff8 is just below a page boundary, which is consistent with a write into the guard page placed under the stack. The hypothesis can be checked by reading the frame size of the function stage in the file user/primes.asm, generated by the build (the instruction addi sp,sp,-N), and by comparing 58 frames with 4096 bytes.

After the experiment, the limit 35 was restored, as shown in figure 3.7, so the delivered file contains the correct version.

```
prime 241 (pid 58)
prime 251 (pid 59)
prime 257 (pid 60)
prime 263 (pid 61)
prime 269 (pid 62)
prime usertrap(): unexpected scause 0xf pid=63
            sepc=0x502 stval=0x2ff8
$ QEMU: Terminated
ion@ion-vitius:~/xv6-riscv$ sed -i 's/i <= 280/i <= 35/' user/primes.c
sed -i 's/ (pid %d)\\n", p, getpid()/\\n", p/' user/primes.c
grep -n "prime %d\|i <=" user/primes.c
16:  printf("prime %d\n", p);
19:    printf("primes: pipe failed after prime %d\n", p);
25:    printf("primes: fork failed after prime %d\n", p);
60:  for(int i = 2; i <= 35; i++)
ion@ion-vitius:~/xv6-riscv$
```

Figure 3.7 Restored limit 35 in user/primes.c

The answer to the second observation question of primes is the following: with the limit 280 the program does not complete, it prints primes up to 269 and then a stage is killed by a page fault; the failure was not caused by an error returned from pipe() or fork(), and the cause remains to be confirmed.

CONCLUSION

Both programs required by the practical work were implemented and run in xv6. The program pingpong prints the lines “received ping” and “received pong” in the required order, using two pipes and closing the unused ends. The program primes prints the eleven primes from 2 to 31 and terminates, which shows that every stage closes its pipe ends and waits for its child [1].

The experiment with the limit 280 did not reproduce the behaviour described in the handout. The program stopped after the prime 269, by a page fault in the 58th stage, without a failure of pipe() or fork(), and the counts of processes and descriptors stay below NPROC, NOFILE and the number of stages predicted for NFILE. The most plausible explanation is the growth of the user stack along the chain of nested stage() calls, and it can be verified by comparing the frame size of stage with the 4096 bytes of the stack [3].

The work confirmed that a pipe is a unidirectional stream whose end of data depends on the closing of every write end, and that a long chain of processes is limited by kernel resources and by the memory layout of each process, not only by the logic of the algorithm.

REFERENCES

[1] Operating Systems, Lab 8: Processes Talking Through Pipes, in xv6. FCIM / FAF, Technical University of Moldova, 2026–2027. Course handout.

[2] MIT 6.1810 Operating System Engineering, Lab: Xv6 and Unix utilities (pingpong, primes). [online]. Available: https://pdos.csail.mit.edu/6.828/2024/labs/util.html

[3] R. Cox, F. Kaashoek, R. Morris. xv6: a simple, Unix-like teaching operating system (RISC-V edition). MIT, 2024. [online]. Available: https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf

[4] R. Cox. Bell Labs and CSP Threads (the prime sieve of Doug McIlroy). [online]. Available: https://swtch.com/~rsc/thread/

[5] xv6-riscv source code, file kernel/param.h. [online]. Available: https://github.com/mit-pdos/xv6-riscv